

MAD & PWA Lab

Journal

Experiment No.	09
Experiment Title.	To implement Service worker events like fetch, sync and push for E-commerce PWA
Roll No.	02
Name	Vanshika Ambwani
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

EXPERIMENT NO. 9

Aim: To implement Service worker events like fetch, sync and push for E-commerce PWA.

Theory:

Service Worker

Service Worker is a script that works on browser background without user interaction independently. Also, It resembles a proxy that works on the user side. With this script, you can track network traffic of the page, manage push notifications and develop “offline first” web applications with Cache API.

Things to note about Service Worker:

- A service worker is a programmable network proxy that lets you control how network requests from your page are handled.
- Service workers only run over HTTPS. Because service workers can intercept network requests and modify responses, "man-in-the-middle" attacks could be very bad.
- The service worker becomes idle when not in use and restarts when it's next needed. You cannot rely on a global state persisting between events. If there is information that you need to persist and reuse across restarts, you can use IndexedDB databases.
- Service workers make extensive use of promises, so if you're new to promises, then you should stop reading this and check out Promises, an introduction.

Fetch Event

You can track and manage page network traffic with this event. You can check existing cache, manage “cache first” and “network first” requests and return a response that you want.

Of course, you can use many different methods but you can find in the following example a “cache first” and “network first” approach. In this example, if the request's and current location's origin are the same (Static content is requested.), this is called “cacheFirst” but if you request a targeted external URL, this is called “networkFirst”.

- **CacheFirst** - In this function, if the received request has cached before, the cached response is returned to the page. But if not, a new response requested from the network.
- **NetworkFirst** - In this function, firstly we can try getting an updated response from the network, if this process completed successfully, the new response will be cached and returned. But if this process fails, we check whether the request has been cached before or not. If a cache exists, it is returned to the page, but if not, this is up to you. You can return dummy content or information messages to the page.

```
self.addEventListener("fetch", function (event) {
  const req = event.request;
  const url = new URL(req.url);

  if (url.origin === location.origin) {
    event.respondWith(cacheFirst(req));
  }
  else {
    event.respondWith(networkFirst(req));
  }
});

async function cacheFirst(req) {
  return await caches.match(req) || fetch(req);
}

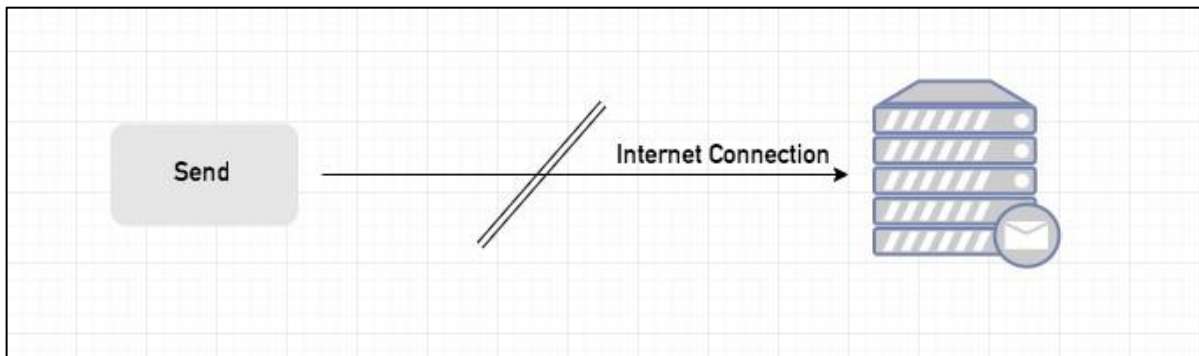
async function networkFirst(req) {
  const cache = await caches.open("pwa-dynamic");
  try {
    const res = await fetch(req);
    cache.put(req, res.clone());
    return res;
  } catch (error) {
    const cachedResponse = await cache.match(req);
    return cachedResponse || await caches.match("./noconnection.json");
  }
}
```

Sync Event

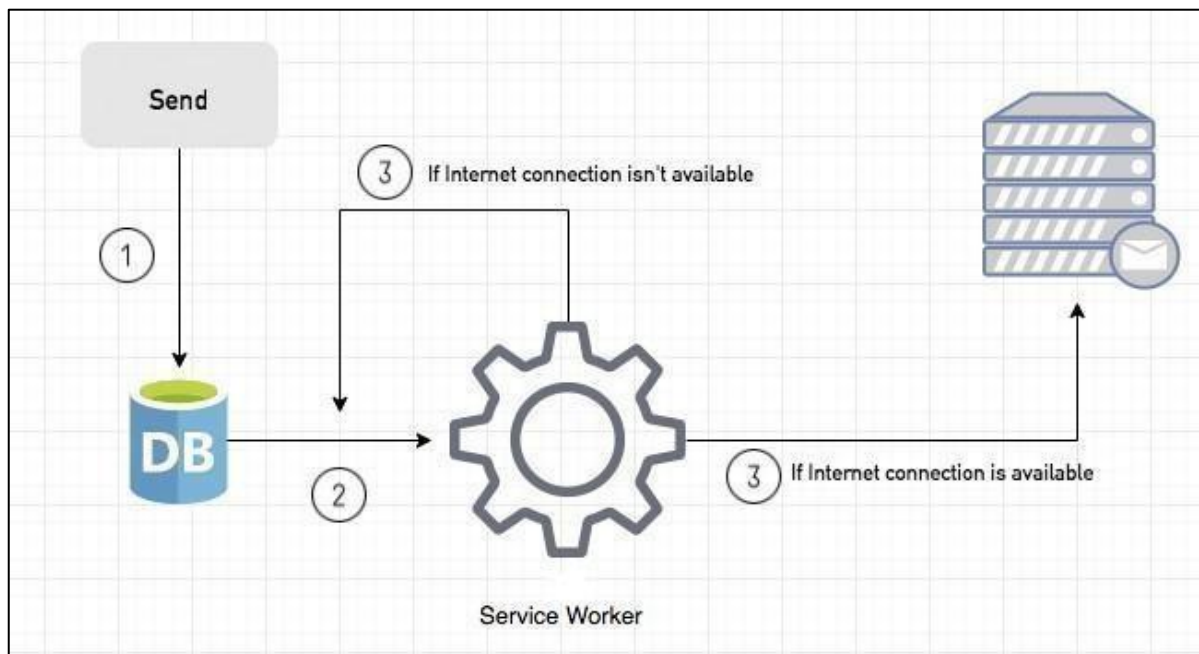
Background Sync is a Web API that is used to delay a process until the Internet connection is stable. We can adapt this definition to the real world; there is an e-mail client application that works on the browser and we want to send an email with this tool. Internet connection is broken while we are writing e-mail content and we didn't realize it. When completing the writing, we click the send button.

Here is a job for the Background Sync.

The following view shows the classical process of sending email to us. If the Internet Connection is broken, we can't send any content to Mail Server.



Here, you can create any scenario for yourself. A sample is in the following for this case.



1. When we click the "send" button, email content will be saved to IndexedDB.
2. Background Sync registration.
3. **If the Internet connection is available**, all email content will be read and sent to Mail Server. **If the Internet connection is unavailable**, the service worker waits until the connection is available even though the window is closed. When it is available, email content will be sent to Mail Server.

You can see the working process within the following code block.

Event Listener for Background Sync Registration

```
document.querySelector("button").addEventListener("click", async () => {
  var swRegistration = await navigator.serviceWorker.register("sw.js");
  swRegistration.sync.register("helloSync").then(function () {
    console.log("helloSync success [main.js]");
  });
});
```

Event Listener for sw.js

```
self.addEventListener('sync', event => {  
  if (event.tag == 'helloSync') {  
    console.log("helloSync [sw.js]");  
  }  
});
```

Push Event

This is the event that handles push notifications that are received from the server. You can apply any method with received data.

We can check in the following example.

“Notification.requestPermission();” is the necessary line to show notification to the user. If you don’t want to show any notification, you don’t need this line.

In the following code block is in sw.js file. You can handle push notifications with this event. In this example, I kept it simple. We send an object that has “method” and “message” properties. If the method value is “pushMessage”, we open the information notification with the “message” property.

```
self.addEventListener('push', event => {  
  if (event && event.data) {  
    var data = event.data.json();  
    if (data.method === "pushMessage") {  
      event.waitUntil(self.registration.showNotification("Test App", {  
        body: data.message  
      }));  
    }  
  }  
});
```

You can use Application Tab from Chrome Developer Tools for testing push notification.

Code:

1. Push Notification code:

```
self.addEventListener("install", (event) => {
  console.log("✅ Service Worker Installed");
});

self.addEventListener("fetch", (event) => {
  event.respondWith(fetch(event.request));
});

// ✅ Background Sync Event
self.addEventListener("sync", (event) => {
  console.log("🔄 Sync Event Triggered:", event.tag); if
  (event.tag === "sync-cart") {
    event.waitUntil(syncCartData());
  }
});

async function syncCartData() {
  try {
    console.log("🛒 Syncing cart data...");
    // Add your actual sync logic here
  } catch (err) {
    console.error("❌ Failed to sync cart data:", err);
  }
}

self.addEventListener("message", (event) => { if
  (event.data && event.data.type === "trigger-push") {
    const { title, body } = event.data; if
    (Notification.permission === "granted") {
      self.registration.showNotification(title, {
        body,
        icon: "/images/banner-1.png",
        badge: "/images/"
      });
    }
  }
});

// ✅ Push Notification Event self.addEventListener("push",
(event) => {
  console.log("📬 Push Event Received");

  let data = {}; if
  (event.data) {
    data = event.data.json();
  }
});
```

```
}

const title = data.title || "New Notification!";
const options = {
  body: data.body || "You have a new message.",
  icon: "/images/logo.png",
  badge: "/images/logo.png"
};

// Check permission before showing notification
if (Notification.permission === "granted") {
  event.waitUntil(
    self.registration.showNotification(title, options)
  );
} else {
  console.warn("⊗ Notification not shown. Permission not granted.");
}
});
```

2. Fetch and Sync Code:

```
3. 'use strict';
4.
5.  /**
6.   * Add event on element
7.   */
8.   const addEventOnElem = function (elem, type, callback) {
9.     if (elem.length > 1) {
10.      for (let i = 0; i < elem.length; i++) { 11.        elem[i].addEventListener(type,
callback);
12.      }
13.    } else {
14.      elem.addEventListener(type, callback);
15.    } 16. };
17.
18.  /**
19.   * Navbar toggle
20.   */
21.   const navTogglers = document.querySelectorAll("[data-nav-toggler]");
22.   const navbar = document.querySelector("[data-navbar]");
23.   const navbarLinks = document.querySelectorAll("[data-nav-link]");
24.   const overlay = document.querySelector("[data-overlay]"); 25.
26.   const toggleNavbar = function () {
27.     navbar.classList.toggle("active");
28.     overlay.classList.toggle("active"); 29. };
30.
31. addEventOnElem(navTogglers, "click", toggleNavbar);
32.
33.   const closeNavbar = function () {
34.     navbar.classList.remove("active");
35.     overlay.classList.remove("active"); 36. };
37.
38. addEventOnElem(navbarLinks, "click", closeNavbar);
39.
40.  /**
41.   * Header sticky & back top btn active
42.   */
43.   const header = document.querySelector("[data-header]");
44.   const backTopBtn = document.querySelector("[data-back-top-btn]"); 45.
46.   const headerActive = function () {
47.     if (window.scrollY > 150) {
48.       header.classList.add("active");
```

```
49. backTopBtn.classList.add("active"); 50. } else {
```

|

|

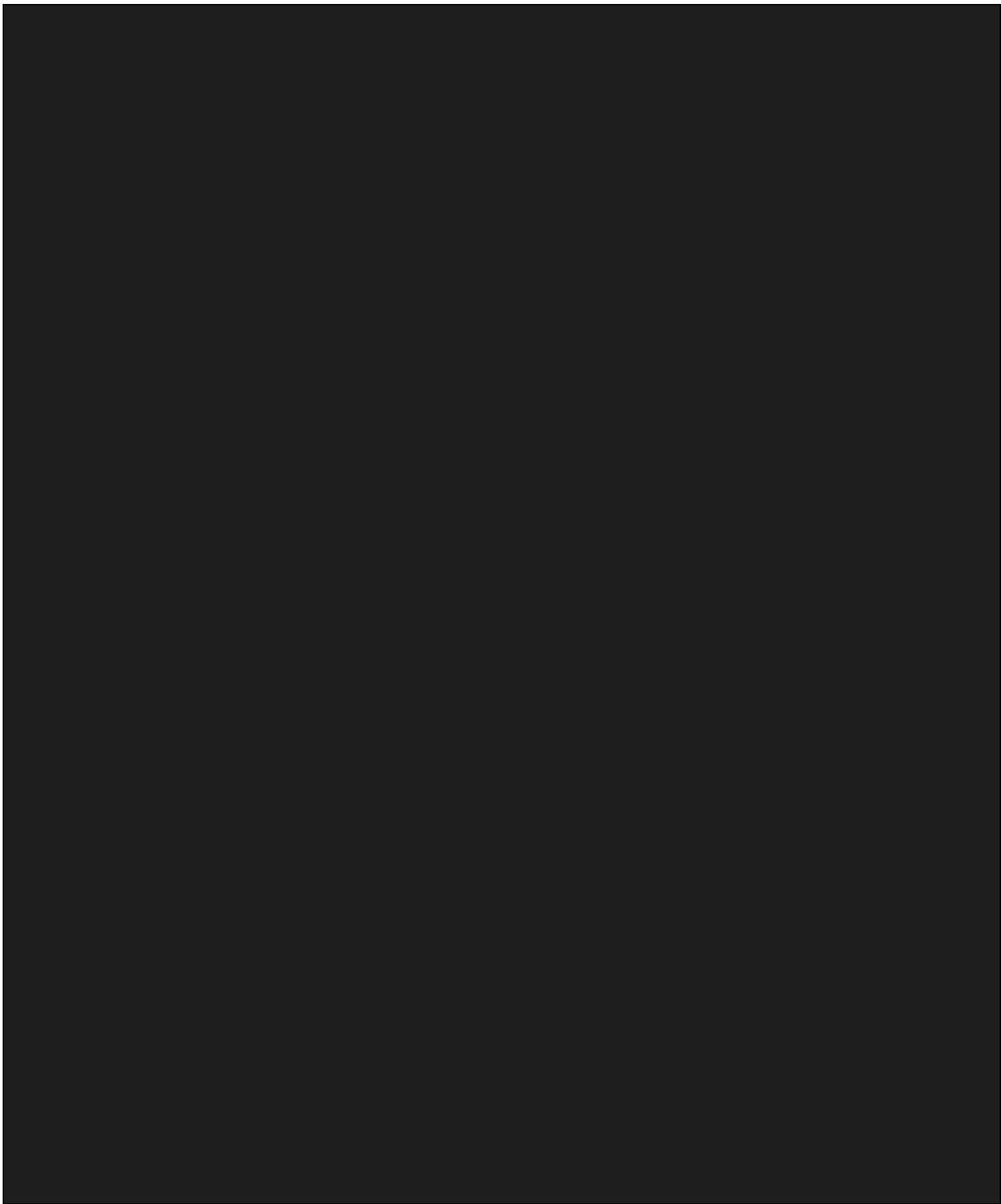
```

51. header.classList.remove("active");
52. backTopBtn.classList.remove("active");
53. } 54. };
55.
56. addEventOnElem(window, "scroll",
headerActive); 57. 58. let lastScrolledPos = 0; 59.
60. const headerSticky = function () {
61. if (lastScrolledPos >= window.scrollY) {
62. header.classList.remove("header-hide");
63. } else {
64. header.classList.add("header-hide"); 65. }
66.
67. lastScrolledPos = window.scrollY; 68.
};
69.
70. addEventOnElem(window, "scroll", headerSticky);
71.
72. /**
73.  * Scroll reveal effect
74.  */
75. const sections = document.querySelectorAll("[data-section]"); 76.
77. const scrollReveal = function () {
78. for (let i = 0; i < sections.length; i++) {
79. if (sections[i].getBoundingClientRect().top < window.innerHeight / 2) {
80. sections[i].classList.add("active");
81. }
82. } 83. };
84.
85. scrollReveal();
86. addEventOnElem(window, "scroll", scrollReveal); 87.
88. /**
89.  *  Register Service Worker and Notification Permission 90. */
91. if ('serviceWorker' in navigator && 'Notification' in window) {
92. window.addEventListener("load", () => {
93. navigator.serviceWorker.register('/service-worker.js') 94.
.then((reg) => {
95. console.log(" Service Worker Registered: ", reg.scope);
96.
97. // Optional: Register background sync
98. if ('sync' in reg) {
99. reg.sync.register('sync-cart')

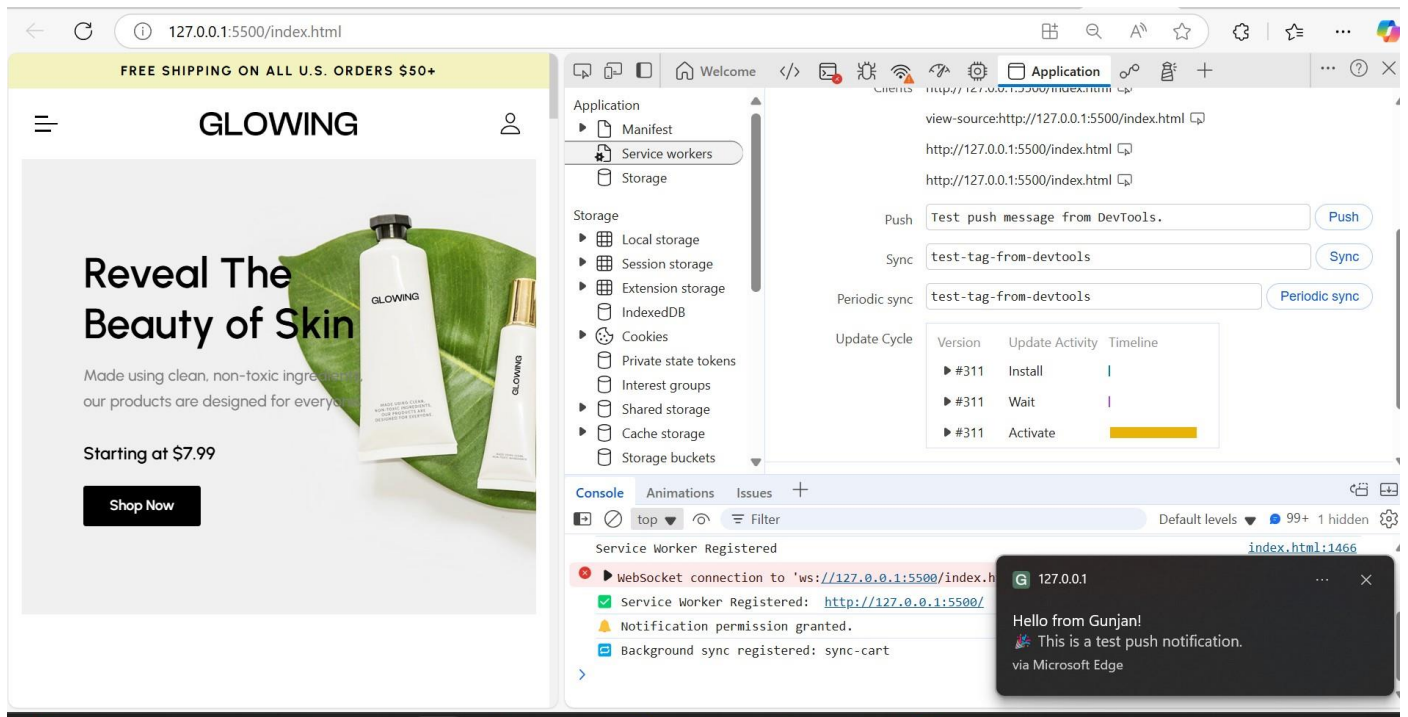
```

```
100.         .then(() => {
```

```
101.         console.log("🔗 Background sync registered: sync-cart"); 102.
102.     })
103.     .catch((err) => {
104.     console.error("❌ Sync registration failed:", err);
105.     }); 106.         } 107.
108.     // Ask for notification permission
109.     Notification.requestPermission().then((permission) => {
110.     if (permission === "granted") {
111.     console.log("🔔 Notification permission granted."); 112.
113.     // Optional: Trigger test push manually
114.     setTimeout(() => {
115.     reg.active.postMessage({
116.     type: 'trigger-push',
117.     title: 'Hello from Gunjan!',
118.     body: '🍷 This is a test push notification.'
119.     });
120.     }, 3000); // Trigger after 3s
121.     } else {
122.     console.warn("🔔 Notification permission denied.");
123.     }
124.     });
125.     })
126.     .catch((err) => {
127.     console.error("❌ Service Worker registration failed:", err);
128.     });
129.     }); 130. }
131.
```



Output:



✓ Service Worker Registered: <http://127.0.0.1:5500/> [script.js:93](#)

🔔 Notification permission granted. [script.js:109](#)

🔧 Background sync registered: sync-cart [script.js:99](#)

>

127.0.0.1:5500/index.html

Application

Interest groups

Shared storage

Cache storage

Storage buckets

Background services

Back/forward cache

Background fetch

Background sync

Bounce tracking ...

Notifications

Payment handler

Periodic backgrou...

Speculative loads

Push messaging

Reporting API

Application

Show events from other domains

Show events from other storage partitions

#	Timestamp	Event	Origin	Storage Key	Servic...	Instance ID
1.	2025-04-07 1...	Push event disp...	http://127.0.0.1:...	http://127.0.0.1:...	/	
2.	2025-04-07 1...	Push event com...	http://127.0.0.1:...	http://127.0.0.1:...	/	
3.	2025-04-07 1...	Push event disp...	http://127.0.0.1:...	http://127.0.0.1:...	/	
4.	2025-04-07 1...	Push event com...	http://127.0.0.1:...	http://127.0.0.1:...	/	
5.	2025-04-07 1...	Push event disp...	http://127.0.0.1:...	http://127.0.0.1:...	/	

No event selected

Select an event to view its metadata

Console

Animations

Issues

Filter

Default levels

83 1 hidden

[NEW] Explain console errors by using Copilot in Edge: click [u](#) to explain an error. [Learn more](#)

WebSocket connection to 'ws://127.0.0.1:5500/index.html/ws' failed: [index.html:1500](#)

Service Worker Registered: [http://127.0.0.1:5500/](#)

Notification permission granted.

Background sync registered: sync-cart

Shop Now

FREE SHIPPING ON ALL U.S. ORDERS \$50+

GLOWING

Reveal The Beauty of Skin

Made using clean, non-toxic ingredients, our products are designed for everyone.

Starting at \$7.99

Shop Now

